

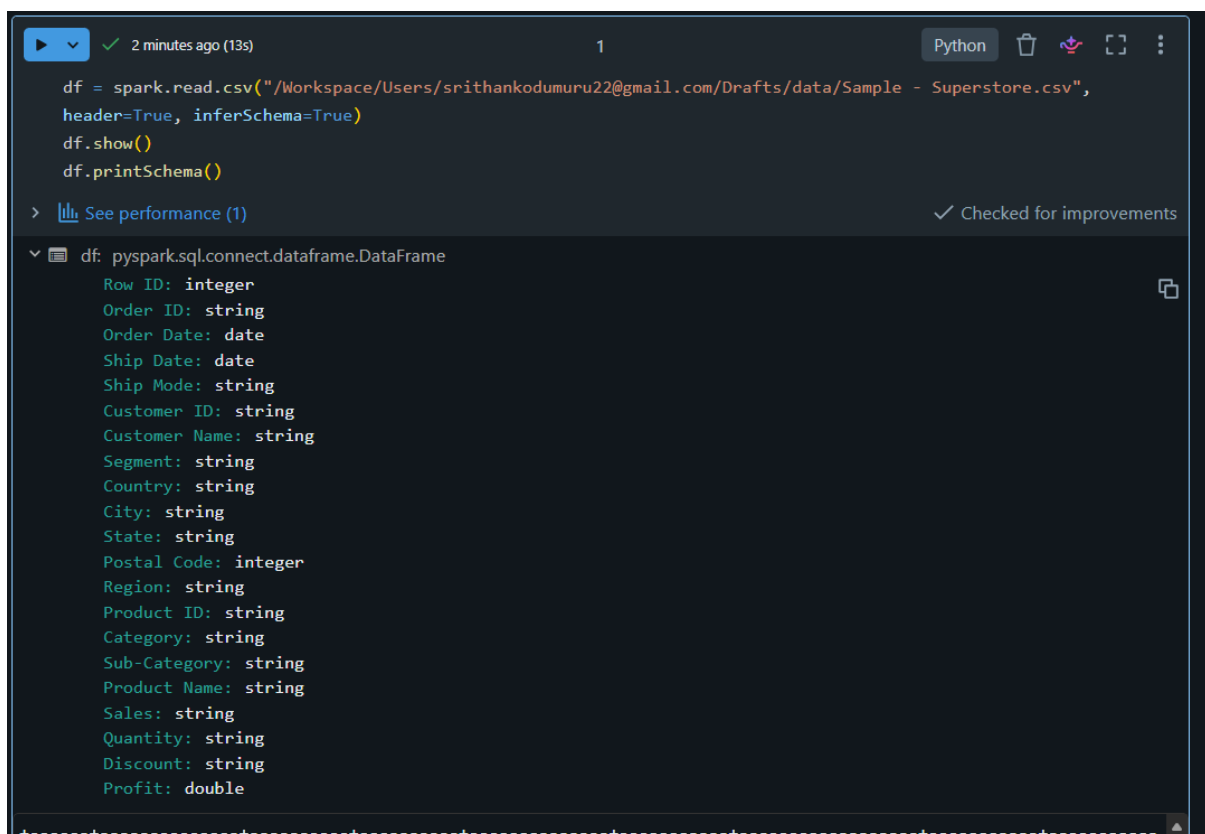
Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer: The Driver is the main part of a Spark application. It creates the Spark session, manages the program, and sends tasks to the executors. The Cluster Manager manages the cluster resources and assigns them to the Spark application. The Executors are the worker processes that run the tasks given by the Driver and process the data. They also store intermediate data during execution.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer: Spark does not execute the code immediately. It first stores all the transformations and waits until an action like `show()` or `collect()` is called. This helps Spark optimize the execution and avoid unnecessary work, making data processing faster and more efficient.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

A screenshot of a Jupyter Notebook interface. At the top, there's a toolbar with a play button, a checkmark, a timestamp '2 minutes ago (13s)', a cell number '1', and a 'Python' language selector. Below the toolbar, the code cell contains the following Python code:

```
df = spark.read.csv("/Workspace/Users/srithankodumuru22@gmail.com/Drafts/data/Sample - Superstore.csv",
header=True, inferSchema=True)
df.show()
df.printSchema()
```

Below the code cell, there's a link 'See performance (1)' and a checkmark with the text 'Checked for improvements'. The output of the code is displayed below the cell, showing the schema of the DataFrame 'df'. The schema is listed as follows:

```
pyspark.sql.connect.dataframe.DataFrame
Row ID: integer
Order ID: string
Order Date: date
Ship Date: date
Ship Mode: string
Customer ID: string
Customer Name: string
Segment: string
Country: string
City: string
State: string
Postal Code: integer
Region: string
Product ID: string
Category: string
Sub-Category: string
Product Name: string
Sales: string
Quantity: string
Discount: string
Profit: double
```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer : CSV stores data row by row, while Parquet stores data column by column. Parquet is faster because Spark reads only the required columns instead of the whole file. It also uses compression, which reduces storage space and improves performance.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```

▶ Just now (2s) 2 Python
df.select("Product ID","Sales")\
  .filter(df["Category"]=="Technology")\
  .show()

> See performance (1) ✓ Checked for improvements

+-----+-----+
| Product ID | Sales |
+-----+-----+
| TEC-PH-10002275 | 907.152 |
| TEC-PH-10002033 | 911.424 |
| TEC-PH-10001949 | 213.48 |
| TEC-AC-10003027 | 90.57 |
| TEC-PH-10004977 | 1097.544 |
| TEC-PH-10000486 | 371.168 |
| TEC-PH-10004093 | 147.168 |
| TEC-AC-10000171 | 45.98 |
| TEC-AC-10002167 | 45 |
| TEC-PH-10003988 | 21.8 |
| TEC-PH-10002447 | 1029.95 |
| TEC-AC-10002167 | 30 |
| TEC-AC-10004633 | 13.98 |
| TEC-PH-10002726 | 167.968 |
| TEC-AC-10001998 | 19.99 |
| TEC-PH-10004093 | 73.584 |
| TEC-AC-10001767 | 95.976 |
| TEC-AC-10001552 | 238.896 |

```

Note: Superstore dataset does not have an Electronics category. It uses categories like Technology, Furniture, and Office Supplies, so Technology is the correct value to filter.

Q6. Write the code to revise a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

```

▶ 1 minute ago (2s) 3 Python
from pyspark.sql.functions import col
df=df.withColumnRenamed("Sales","Total Sales")\
  .withColumn("Total Sales",col("Total Sales").try_cast("double"))
df.show(5,truncate=False)

> See performance (1) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 19 more fields]

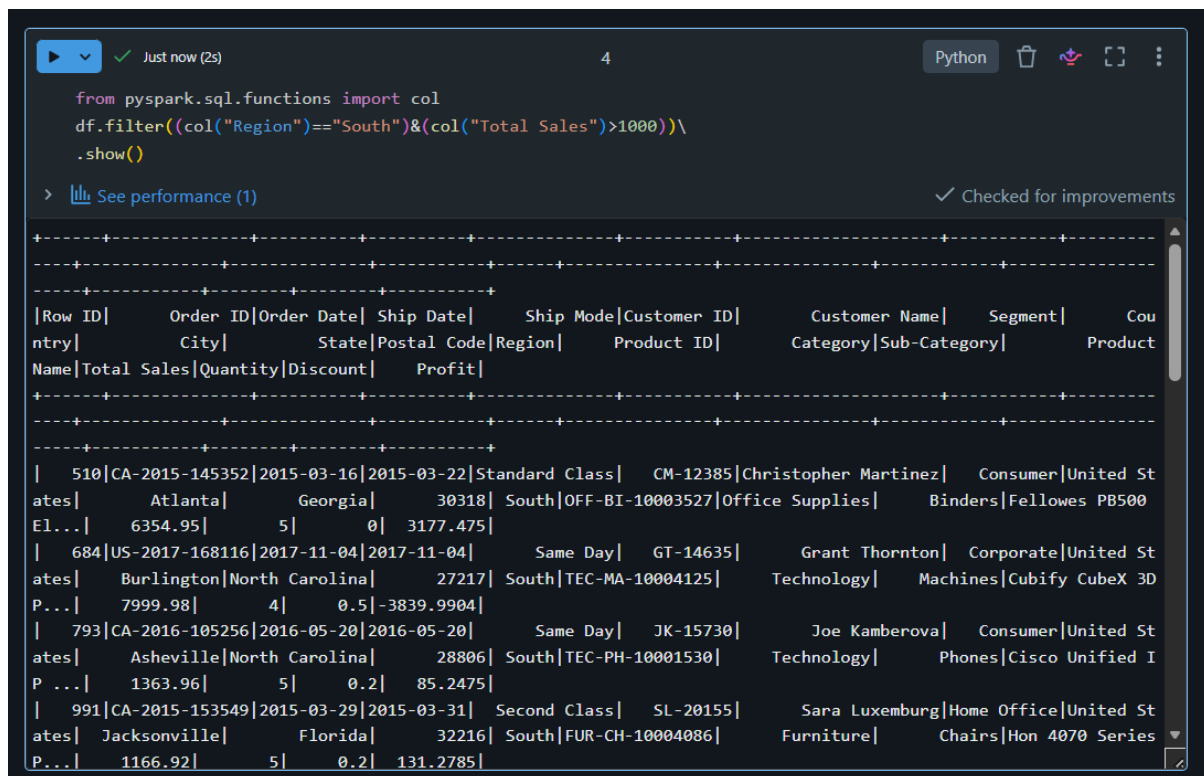
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Row ID|Order ID|Order Date|Ship Date|Ship Mode|Customer ID|Customer Name|Segment|Country|City|
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Total Sales|Quantity|Discount|Profit|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1|CA-2016-152156|2016-11-08|2016-11-11|Second Class|CG-12520|Claire Gute|Consumer|United States|Henderson|
|Kentucky|42420|South|FUR-B0-10001798|Furniture|Bookcases|Bush Somerset Collection|41.9136|
|2|CA-2016-152156|2016-11-08|2016-11-11|Second Class|CG-12520|Claire Gute|Consumer|United States|Henderson|
|Kentucky|42420|South|FUR-CH-10000454|Furniture|Chairs|Hon Deluxe Fabric Upholstered Stacking Chairs, Rounded Back|731.94|
|3|3|0|219.582|
|3|CA-2016-138688|2016-06-12|2016-06-16|Second Class|DV-13045|Darrin Van Huff|Corporate|United States|Los Angeles|
|California|90036|West|OFF-LA-10000240|Office Supplies|Labels|Self-Adhesive Address Labels for Typewriters by Universal|14.62|
|4|2|0|6.8714|
|4|US-2015-108966|2015-10-11|2015-10-18|Standard Class|SO-20335|Sean O'Donnell|Consumer|United States|Fort Lauderdale|
|Florida|33311|South|FUR-TA-10000577|Furniture|Tables|Bretford CR4500 Series S11im Rectangular Table|
|957.5775|5|0.45|-383.031|

```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Answer : Spark keeps a record of all the transformations in a Lineage Graph (DAG). If a worker node fails and some data is lost, Spark uses this graph to recompute only the missing data instead of processing the entire dataset again. This provides fault tolerance and saves time.

Q8. Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.



The screenshot shows a Databricks notebook interface. At the top, there's a toolbar with a play button, a checkmark, and the text 'Just now (2s)'. Below this, the code cell contains the following PySpark code:

```
from pyspark.sql.functions import col
df.filter((col("Region")=="South")&(col("Total Sales")>1000))\
.show()
```

Below the code, there's a link 'See performance (1)' and a status 'Checked for improvements'. The output of the query is displayed as a table with the following columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Product Category, Sub-Category, Product Name, Total Sales, Quantity, Discount, Profit. The table shows several rows of data, including orders from Atlanta, Georgia, Burlington, North Carolina, Asheville, North Carolina, and Jacksonville, Florida.

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer: Predicate Pushdown is a feature in Parquet where Spark applies filter conditions while reading the file. It reads only the required data instead of the entire dataset. This reduces memory usage, decreases disk I/O, and improves performance.

Q10. Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

```

from pyspark.sql.functions import col

df=df.withColumn("Final Price",col("Total Sales")*1.18)
df.show(5)

```

> [See performance \(1\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 20 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Total Sales	Quantity	Discount	Profit	Final Price	
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-B0-10001798	Furniture	Bookcases	Bush Somerset Col...	26	1.96	2	0	41.9136	309.11279999999994
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric...	73	1.94	3	0	219.582	863.6892
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Add...	14.62	2	0	6.8714	17.2516	
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 S...	9	57.5775	5	0.45	-383.031	1129.94145

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Answer: Transformations are operations that change the data or create a new dataframe , but Spark does not execute them immediately. It waits until an action is called. Actions are operations that execute all the transformations and return the result.

Examples of Transformations: filter(), select()

Examples of Actions: show(), collect()

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```

df.write.mode("overwrite").parquet("/Volumes/workspace/srithankodumuru22/spark_volume/parquet_data")

```

> [See performance \(1\)](#) ✓ Checked for improvements

```

spark.read.parquet("/Volumes/workspace/srithankodumuru22/spark_volume/parquet_data").\
filter("`Customer ID` IS NOT NULL").write.mode("overwrite")\
.option("header", True) \
.csv("/Volumes/workspace/srithankodumuru22/spark_volume/output_csv")

```

> [See performance \(1\)](#) ✓ Checked for improvements

```
spark.read.parquet("/Volumes/workspace/srithankodumuru22/spark_volume/parquet_data").show(5)
```

> [See performance \(1\)](#) ✓ Checked for improvements

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	
Sales	Quantity	Discount	Profit					
4579	CA-2015-110947	2015-06-09	2015-06-15	Standard Class	AG-10765	Anthony Garverick	Home Office	United States
Hampton	Virginia	23666	South	OFF-BI-10001670	Office Supplies	Binders	Vinyl Sectional P...	
113.1	3	0	56.55					
4580	US-2014-150126	2014-07-27	2014-08-02	Standard Class	AS-10045	Aaron Smayling	Corporate	United States
New York City	New York	10035	East	OFF-PA-10002709	Office Supplies	Paper	Xerox 1956	
65.78	11	0	32.2322					
4581	CA-2015-164427	2015-07-31	2015-08-06	Standard Class	AR-10405	Allen Rosenblatt	Corporate	United States
Hattiesburg	Mississippi	39401	South	TEC-AC-10004469	Technology	Accessories	Microsoft Sculpt ...	
239.7	6	0	105.468					
4582	CA-2016-120250	2016-09-19	2016-09-22	First Class	AP-10720	Anne Pryor	Home Office	United States
Philadelphia	Pennsylvania	19140	East	FUR-FU-10003424	Furniture	Furnishings	Nu-Dell Oak Frame	
25.632	3	0.2	3.8448					

```
spark.read.option("header", True).csv("/Volumes/workspace/srithankodumuru22/spark_volume/output_csv").show(5)
```

> [See performance \(1\)](#) ✓ Checked for improvements

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	
Sales	Quantity	Discount	Profit					
4579	CA-2015-110947	2015-06-09	2015-06-15	Standard Class	AG-10765	Anthony Garverick	Home Office	United States
Hampton	Virginia	23666	South	OFF-BI-10001670	Office Supplies	Binders	Vinyl Sectional P...	
113.1	3	0	56.55					
4580	US-2014-150126	2014-07-27	2014-08-02	Standard Class	AS-10045	Aaron Smayling	Corporate	United States
New York City	New York	10035	East	OFF-PA-10002709	Office Supplies	Paper	Xerox 1956	
65.78	11	0	32.2322					
4581	CA-2015-164427	2015-07-31	2015-08-06	Standard Class	AR-10405	Allen Rosenblatt	Corporate	United States
Hattiesburg	Mississippi	39401	South	TEC-AC-10004469	Technology	Accessories	Microsoft Sculpt ...	
239.7	6	0	105.468					
4582	CA-2016-120250	2016-09-19	2016-09-22	First Class	AP-10720	Anne Pryor	Home Office	United States
Philadelphia	Pennsylvania	19140	East	FUR-FU-10003424	Furniture	Furnishings	Nu-Dell Oak Frame	
25.632	3	0.2	3.8448					
4583	CA-2016-121993	2016-06-10	2016-06-12	First Class	JB-16000	Joy Bell-	Consumer	United States
Philadelphia	Pennsylvania	19140	East	OFF-LA-10003720	Office Supplies	Labels	Avery 487	

Implementation: Initially, I encountered a storage permission issue while saving the output files. After identifying the correct writable location, I created and used a Databricks Volume (spark_volume). I successfully saved the dataset as a Parquet file, read it back, filtered the required records, and saved the final output as a CSV file.

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Answer : In Client Mode, the Driver program runs on the local machine where the Spark application is submitted. In Cluster Mode, the Driver runs inside the cluster along with the executors. Client Mode is mainly used for development and testing, while Cluster Mode is preferred for running production applications.

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
from pyspark.sql.functions import col

df.filter((col("Region") == "North") | (col("Segment") == "Consumer")) \
.show()
```

> [See performance \(1\)](#) ✓ Checked for improvements

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Total Sales
Quantity	Discount	Profit	Final Price					
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Col...	261.96
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States
Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric...	731.94
3	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	Bretford CR4500	957.5775
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States
Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage	Eldon Fold 'N Ro	22.368

Q15. When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer : .show(5) displays only the first 5 rows of the dataset, so it uses very little memory. On the other hand, .collect() loads the entire dataset into the Driver's memory. If the dataset is very large, it can cause memory issues or even crash the application. That's why .show(5) is safer for exploring large datasets.

Workflow

Read CSV Dataset



Check Schema



Select Required Columns



Filter Data



Modify DataFrame



Handle Null Values



Save Output